1. HTTP: Основные понятия

HTTP (HyperText Transfer Protocol) – «протокол передачи гипертекста» – протокол прикладного уровня передачи данных (изначально – в виде гипертекстовых документов в формате HTML, в настоящий момент используется для передачи произвольных данных).

Основой HTTP является технология «клиент-сервер»:

– Клиенты – потребители, которые инициируют соединение и посылают запрос.

– Серверы – поставщики, которые ожидают соединения для получения запроса, производят необходимые действия и возвращают обратно сообщение с результатом.

Основной объект манипуляции в HTTP – ресурс, на который указывает URI (Uniform Resource Identifier) в запросе клиента. Обычно это хранящиеся на сервере файлы, но ими могут быть логические объекты или что-то абстрактное.

Особенность протокола: возможность указать в запросе и ответе способ представления одного и того же ресурса по различным параметрам: формату, кодировке, языку и т. д. (для этого используется HTTP-заголовок). Благодаря этому клиент и сервер могут обмениваться двоичными данными, хотя протокол является текстовым.

URI – отвечает на вопрос: «Где и как найти что-то?» Пример: http://example.com/just/some/long/path/path

2. HTTP: История версий

HTTP/0.9

– Предложен в марте 1991 года Тимом Бернерсом-Ли (CERN) как механизм доступа к документам в Интернете и облегчения навигации посредством гипертекста.

– Самая ранняя версия впервые опубликована в январе 1992 г. (реализация датируется 1990 годом).

– Спецификация привела к упорядочению правил взаимодействия между клиентами и серверами, а также чёткому разделению функций между этими компонентами.

– Задокументированы основные синтаксические и семантические положения.

HTTP/1.0

– В мае 1996 года выпущен информационный документ RFC 1945 – основа для реализации большинства компонентов HTTP/1.0.

HTTP/1.1

– Текущая версия, принята в июне 1999 года.

– Новое: режим «постоянного соединения» – TCP-соединение может оставаться открытым после отправки ответа на запрос, что позволяет посылать несколько запросов за одно соединение.

– Клиент обязан посылать информацию об имени хоста, к которому он обращается, что сделало возможной более простую организацию виртуального хостинга.

HTTP/2

– 11 февраля 2015 года опубликованы финальные версии черновика следующей версии протокола.

– В отличие от предыдущих версий, протокол HTTP/2 является бинарным.

– Ключевые особенности: мультиплексирование запросов, расстановка приоритетов для запросов, сжатие заголовков, загрузка нескольких элементов параллельно посредством одного TCP-соединения, поддержка проактивных push-уведомлений со стороны сервера.

RFC – документы, содержащие технические спецификации и стандарты, широко применяемые во всемирной сети интернет. Существует более 5000 документов.

3. HTTP: Структура сообщения

Каждое HTTP-сообщение состоит из трёх частей, которые передаются в указанном порядке:

1. Стартовая строка (Starting line) – определяет тип сообщения.

2. Заголовки (Headers) – характеризуют тело сообщения, параметры передачи и прочие сведения.

3. Тело сообщения (Message Body) – непосредственно данные сообщения. Обязательно должно отделяться от заголовков пустой строкой.

Заголовки и тело сообщения могут отсутствовать, но стартовая строка является обязательным элементом, так как указывает на тип запроса/ответа.

Исключение: версия 0.9 протокола – сообщение запроса содержит только стартовую строку, а сообщения ответа только тело сообщения.

Для версии 1.1 сообщение запроса обязательно должно содержать заголовок Host.

4. HTTP: Стартовые строки

Стартовая строка запроса:

```
text
Метод URI HTTP/Версия
```

– Метод (Method) – название запроса, одно слово заглавными буквами.

– URI – определяет путь к запрашиваемому документу.

– Версия (Version) – пара разделённых точкой цифр. Например: 1.0.

Стартовая строка ответа:

text

HTTP/Версия Код-Состояния Пояснение

– Версия – пара разделённых точкой цифр, как в запросе.

– Код состояния (Status Code) – три цифры. По коду состояния определяется дальнейшее содержимое сообщения и поведение клиента.

– Пояснение (Reason Phrase) – текстовое короткое пояснение к коду ответа для пользователя. Никак не влияет на сообщение и является необязательным.

Пример HTTP-обмена:

```
text
GET / HTTP/1.1
Host: xbb.uz
User-Agent: Mozilla/5.0 ...
Accept: text/html,applic ...
Accept-Language: ru-ru,r ...
Accept-Encoding: gzip,de ...
text
HTTP/1.0 200 OK
Server: nginx/0.7.67
Date: Tue, 08 Feb 2011 08: ...
Content-Type: text/html; c ...
X-Powered-By: PHP/5.2.12
Expires: Thu, 19 Nov 1981 ...
```

5. HTTP: Методы

Метод – последовательность из любых символов, кроме управляющих и разделителей, указывающая на основную операцию над ресурсом. Метод представляет собой короткое английское слово, записанное заглавными буквами. Название метода чувствительно к регистру.

Каждый сервер обязан поддерживать как минимум методы GET и HEAD.

– Если сервер не распознал указанный клиентом метод, он должен вернуть статус 501 (Not Implemented).

– Если серверу метод известен, но он неприменим к конкретному ресурсу, возвращается сообщение с кодом 405 (Method Not Allowed).

– В обоих случаях серверу следует включить в сообщение ответа заголовок Allow со списком поддерживаемых методов.

Основные методы: OPTIONS, GET, HEAD, POST, PUT, PATCH, DELETE, TRACE, CONNECT.

OPTIONS

Используется для определения возможностей веб-сервера или параметров соединения для конкретного ресурса. В ответ серверу следует включить заголовок Allow со списком поддерживаемых методов. Также в заголовке ответа может включаться информация о поддерживаемых расширениях. Результат выполнения этого метода не кэшируется.

GET

Используется для запроса содержимого указанного ресурса. С помощью метода GET можно также начать какой-либо процесс. В этом случае в тело ответного сообщения следует включить информацию о ходе выполнения процесса.

Клиент может передавать параметры выполнения запроса в URI целевого ресурса после символа «?»:

```
text
GET /path/resource?param1=value1&param2=value2 HTTP/1.1
```

Согласно стандарту HTTP, запросы типа GET считаются идемпотентными.

HEAD

Аналогичен методу GET, за исключением того, что в ответе сервера отсутствует тело. Запрос HEAD обычно применяется для извлечения метаданных, проверки наличия ресурса (валидация URL) и чтобы узнать, не изменился ли он с момента последнего обращения.

Заголовки ответа могут кэшироваться. При несовпадении метаданных ресурса с соответствующей информацией в кэше копия ресурса помечается как устаревшая.

POST

Применяется для передачи пользовательских данных заданному ресурсу. Например, в блогах посетители обычно могут вводить свои комментарии к записям в HTML-форму, после чего они передаются серверу методом POST и он помещает их на страницу. При этом передаваемые данные (в примере с блогами – текст комментария) включаются в тело запроса. Аналогично с помощью метода POST обычно загружаются файлы на сервер.

В отличие от метода GET, метод POST не считается идемпотентным, то есть многократное повторение одних и тех же запросов POST может возвращать разные результаты (например, после каждой отправки комментария будет появляться очередная копия этого комментария).

При результате выполнения 200 (Ok) в тело ответа следует включить сообщение об итоге выполнения запроса. Если был создан ресурс, то серверу следует вернуть ответ 201 (Created) с указанием URI нового ресурса в заголовке Location.

Сообщение ответа сервера на выполнение метода POST не кэшируется.

PUT

Применяется для загрузки содержимого запроса на указанный в запросе URI. Если по заданному URI не существует ресурс, то сервер создаёт его и возвращает статус 201 (Created). Если же был изменён ресурс, то сервер возвращает 200 (Ok) или 204 (No Content). Сервер не должен игнорировать некорректные заголовки Content-*, передаваемые клиентом вместе с сообщением. Если какой-то из этих заголовков не может быть распознан или не допустим при текущих условиях, то необходимо вернуть код ошибки 501 (Not Implemented).

Фундаментальное различие методов POST и PUT заключается в понимании предназначений URI ресурсов. Метод POST предполагает, что по указанному URI будет производиться обработка передаваемого клиентом содержимого. Используя PUT, клиент предполагает, что загружаемое содержимое соответствует находящемуся по данному URI ресурсу.

Сообщения ответов сервера на метод PUT не кэшируются.

PATCH

Аналогично PUT, но применяется только к фрагменту ресурса.

DELETE

Удаляет указанный ресурс.

TRACE

Возвращает полученный запрос так, что клиент может увидеть, какую информацию промежуточные серверы добавляют или изменяют в запросе.

CONNECT

Преобразует соединение запроса в прозрачный TCP/IP-туннель, обычно чтобы содействовать установлению защищённого SSL-соединения через нешифрованный прокси.

6. HTTP: Коды состояния

1xx: Informational («Информационный»)

В этот класс выделены коды, информирующие о процессе передачи. В HTTP/1.0 сообщения с такими кодами должны игнорироваться. В HTTP/1.1 клиент должен быть готов принять этот класс сообщений как обычный ответ, но ничего отправлять серверу не нужно. Сами сообщения от сервера содержат только стартовую строку ответа и, если требуется, несколько специфичных для ответа полей заголовка. Прокси-серверы подобные сообщения должны отправлять дальше от сервера к клиенту.

- 100 Continue («продолжай»)
- 101 Switching Protocols («переключение протоколов»)
- 102 Processing («идёт обработка»)

2xx: Success («Успех»)

Сообщения данного класса информируют о случаях успешного принятия и обработки запроса клиента. В зависимости от статуса сервер может ещё передать заголовки и тело сообщения.

- 200 OK («хорошо»)
- 201 Created («создано»)
- 202 Accepted («принято»)
- 203 Non-Authoritative Information («информация не авторитетна»)
- 204 No Content («нет содержимого»)
- 205 Reset Content («сбросить содержимое»)
- 206 Partial Content («частичное содержимое»)
- 207 Multi-Status («многостатусный»)

3xx: Redirection («Перенаправление»)

Коды класса 3xx сообщают клиенту, что для успешного выполнения операции необходимо сделать другой запрос (как правило, по другому URI).

Из данного класса пять кодов 301, 302, 303, 305 и 307 относятся непосредственно к перенаправлениям (редирект). Адрес, по которому клиенту следует произвести запрос, сервер указывает в заголовке Location. При этом допускается использование фрагментов в целевом URI.

– 300 Multiple Choices («множество выборов»)

– 301 Moved Permanently («перемещено навсегда»)

– 302 Moved Temporarily («перемещено временно»)

– 302 Found («найдено»)

– 303 See Other (смотреть другое)

– 304 Not Modified (не изменялось)

– 305 Use Proxy («использовать прокси»)

– 306 – зарезервировано (код использовался только в ранних спецификациях)

– 307 Temporary Redirect («временное перенаправление»)

4xx: Client Error («Ошибка клиента»)

Класс кодов 4xx предназначен для указания ошибок со стороны клиента. При использовании всех методов, кроме HEAD, сервер должен вернуть в теле сообщения гипертекстовое пояснение для пользователя.

– 400 Bad Request («плохой, неверный запрос»)

– 401 Unauthorized («не авторизован»)

– 402 Payment Required («необходима оплата»)

– 403 Forbidden («запрещено»)

– 404 Not Found («не найдено»)

– 405 Method Not Allowed («метод не поддерживается»)

– 406 Not Acceptable («неприемлемо»)

– 407 Proxy Authentication Required («необходима аутентификация прокси»)

– 408 Request Timeout («истекло время ожидания»)

– 409 Conflict («конфликт»)

– 410 Gone («удалён»)

– 411 Length Required («необходима длина»)

– 412 Precondition Failed («условие ложно»)

– 413 Request Entity Too Large («размер запроса слишком велик»)

– 414 Request-URI Too Large («запрашиваемый URI слишком длинный»)

– 415 Unsupported Media Type («неподдерживаемый тип данных»)

– 416 Requested Range Not Satisfiable («запрашиваемый диапазон не достижим»)

– 417 Expectation Failed («ожидаемое неприемлемо»)

– 422 Unprocessable Entity («необрабатываемый экземпляр»)

– 423 Locked («заблокировано»)

– 424 Failed Dependency («невыполненная зависимость»)

– 425 Unordered Collection («неупорядоченный набор»)

– 426 Upgrade Required («необходимо обновление»)

– 428 Precondition Required («необходимо предусловие»)

– 429 Too Many Requests («слишком много запросов»)

– 431 Request Header Fields Too Large («поля заголовка запроса слишком большие»)

– 434 Requested host unavailable («Запрашиваемый адрес недоступен»)

– 444 Закрывает соединение без передачи заголовка ответа. Нестандартный код

– 449 Retry With («повторить с»)

– 451 Unavailable For Legal Reasons («недоступно по юридическим причинам»)

5xx: Server Error («Ошибка сервера»)

Коды 5xx выделены под случаи неудачного выполнения операции по вине сервера. Для всех ситуаций, кроме использования метода HEAD, сервер должен включать в тело сообщения объяснение, которое клиент отобразит пользователю.

– 500 Internal Server Error («внутренняя ошибка сервера»)

– 501 Not Implemented («не реализовано»)

– 502 Bad Gateway («плохой, ошибочный шлюз»)

– 503 Service Unavailable («сервис недоступен»)

– 504 Gateway Timeout («шлюз не отвечает»)

– 505 HTTP Version Not Supported («версия HTTP не поддерживается»)

– 506 Variant Also Negotiates («вариант тоже проводит согласование»)

– 507 Insufficient Storage («переполнение хранилища»)

– 508 Loop Detected («обнаружена петля»)

– 509 Bandwidth Limit Exceeded («исчерпана пропускная ширина канала»)

– 510 Not Extended («не расширено»)

– 511 Network Authentication Required («требуется сетевая аутентификация»)

7. HTTP: Заголовки

HTTP Headers – это строки в HTTP-сообщении, содержащие разделённую двоеточием пару параметр-значение. Формат заголовков соответствует общему формату заголовков текстовых сетевых сообщений ARPA (RFC 822). Заголовки должны отделяться от тела сообщения хотя бы одной пустой строкой.

Пример заголовков:

```
text
Server: Apache/2.2.11 (Win32) PHP/5.3.0
Last-Modified: Sat, 16 Jan 2010 21:16:42 GMT
Content-Type: text/plain; charset=windows-1251
Content-Language: ru
```

Все заголовки разделяются на четыре основных группы:

1. General Headers («Основные заголовки») – могут включаться в любое сообщение клиента и сервера.

2. Request Headers («Заголовки запроса») – используются только в запросах клиента.

3. Response Headers («Заголовки ответа») – только для ответов от сервера.

4. Entity Headers («Заголовки сущности») – сопровождают каждую сущность сообщения.

Примеры заголовков:

Заголовок	Описание	Пример
Accept	Список допустимых форматов ресурса	Accept: text/plain
Accept-Charset	Перечень поддерживаемых кодировок для предоставления пользователю	Accept-Charset: utf-8
Allow	Список поддерживаемых методов	Allow: OPTIONS, GET, HEAD
Referer	URI ресурса, после которого клиент сделал текущий запрос	Referer: http://en.wikipedia.org/wiki/Main_Page
User-Agent	Список названий и версий клиента и его компонентов с комментариями	User-Agent: Mozilla/5.0 (X11; Linux i686; rv:2.0.1) Gecko/20100101 Firefox/4.0.1

8. HTTP vs HTTPS

HTTPS (HyperText Transfer Protocol Secure) – расширение протокола HTTP, поддерживающее шифрование. Данные, передаваемые по протоколу HTTPS, «упаковываются» в криптографический протокол SSL или TLS. В отличие от HTTP, для HTTPS по умолчанию используется TCP-порт 443.

HTTPS не является отдельным протоколом. Это обычный HTTP, работающий через шифрованные транспортные механизмы SSL и TLS. Он обеспечивает защиту от атак, основанных на прослушивании сетевого соединения – от снифферских атак и атак типа man-in-the-middle, при условии, что будут использоваться шифрующие средства и сертификат сервера проверен и ему доверяют.

По умолчанию HTTPS URL использует 443 TCP-порт (для незащищённого HTTP – 80). Чтобы подготовить веб-сервер для обработки https-соединений, администратор должен получить и установить в систему сертификат для этого веб-сервера. Сертификат состоит из 2 частей (2 ключей) – public и private. Public-часть сертификата используется для зашифровывания трафика от клиента к серверу в защищённом соединении, private-часть – для расшифровывания полученного от клиента зашифрованного трафика на

сервере. После того как пара ключей приватный/публичный сгенерированы, на основе публичного ключа формируется запрос на сертификат в Центр сертификации, в ответ на который ЦС высылает подписанный сертификат. ЦС при подписании проверяет клиента, что позволяет ему гарантировать, что держатель сертификата является тем, за кого себя выдаёт (обычно это платная услуга).

Традиционно на одном IP-адресе может работать только один HTTPS сайт. Для работы нескольких HTTPS-сайтов с различными сертификатами применяется расширение TLS под названием Server Name Indication (SNI).

9. JSON

JSON (JavaScript Object Notation) – это простой текстовый формат для хранения и передачи данных.

JSON основан на двух структурах данных:

1. Коллекция пар ключ/значение. В разных языках эта концепция реализована как объект, запись, структура, словарь, хэш, именованный список или ассоциативный массив.

2. Упорядоченный список значений. В большинстве языков это реализовано как массив, вектор, список или последовательность.

Это универсальные структуры данных. Почти все современные языки программирования поддерживают их в какой-либо форме.

JSON был создан для передачи данных между сайтами или из сайта. Во многих API есть JSON: Facebook API, VK API и т.д.

Примеры:

– https://samples.openweathermap.org/data/2.5/weather?q=London,uk&appid=b6907d289e10d714a6e88b30761fae22

– https://www.jsonmate.com/

Схема взаимодействия:

```
text
Application <--> API <--> Back-end Server
                      |
                  JSON, XML...
```